

GLOBAL E-COMMERCE PLATFORM

System Architecture Explained

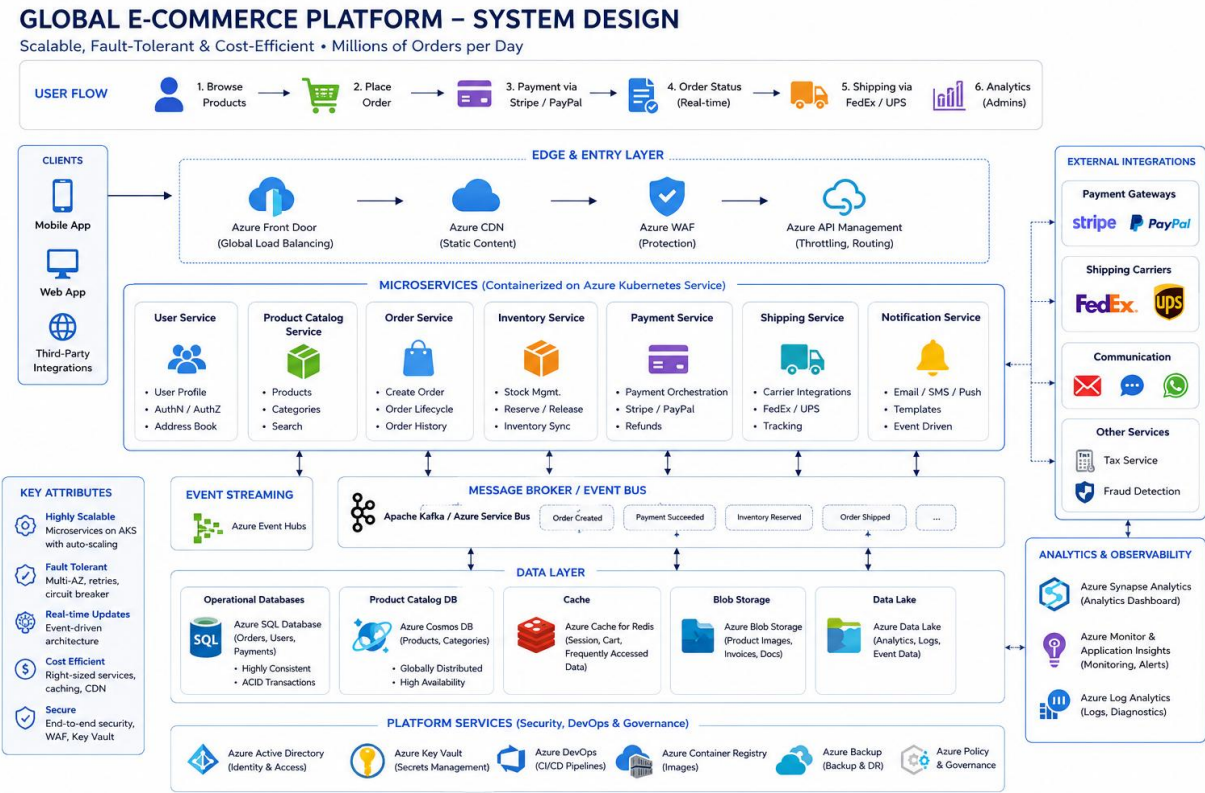


Figure 1: Full system architecture diagram

1. Overview

This document walks through the architecture of a global e-commerce platform designed to handle millions of orders per day. The system is built around a cloud-native, microservices-based approach on Azure Kubernetes Service (AKS), using event-driven communication, polyglot data storage, and managed Azure services for security, observability, and governance.

The design is organized into distinct layers that together take a customer request from the moment they open the app to the moment their order is delivered and analyzed: the client layer, the edge and entry layer, the microservices layer, the event streaming and messaging layer, the data layer, external integrations, analytics and observability, and platform services.

2. End-to-End User Flow

At the top of the diagram, the six-step user journey summarizes what the entire architecture exists to support:

- **1. Browse Products** – the customer explores the catalogue through the mobile app, web app, or a third-party integration.
- **2. Place Order** – the customer adds items to a cart and checks out, triggering order creation.
- **3. Payment via Stripe / PayPal** – payment is authorized and captured through an external payment gateway.
- **4. Order Status (Real-time)** – the customer sees live order updates as the order moves through fulfillment.
- **5. Shipping via FedEx / UPS** – a shipping carrier is engaged to pick, pack, and deliver the order, with tracking exposed back to the customer.
- **6. Analytics (Admins)** – operational and business data generated by the journey feeds dashboards used by internal administrators.

Every subsequent layer in the architecture exists to make one or more of these six steps fast, reliable, and secure at scale.

3. Client Layer

Traffic originates from three types of clients, all of which talk to the platform through the same entry point rather than connecting directly to backend services:

- **Mobile App** – native iOS/Android client used by shoppers.
- **Web App** – browser-based storefront and admin experience.
- **Third-Party Integrations** – partner systems, marketplaces, or affiliate platforms that consume the platform's APIs.

4. Edge & Entry Layer

Before any request reaches a microservice, it passes through four Azure edge services arranged in sequence. This layer protects the backend, absorbs traffic spikes, and reduces unnecessary load:

- **Azure Front Door** – provides global load balancing, routing each request to the closest and healthiest regional deployment.
- **Azure CDN** – caches and serves static content (images, scripts, product assets) close to the user, reducing latency and origin load.
- **Azure WAF (Web Application Firewall)** – inspects traffic and blocks common web exploits and malicious requests before they reach the application.
- **Azure API Management** – handles throttling, routing, versioning, and API key enforcement, acting as the single front door into the microservices layer.

5. Microservices Layer (Azure Kubernetes Service)

The core business logic is split into seven independently deployable microservices, all containerized and run on Azure Kubernetes Service (AKS) so each can scale independently based on its own load pattern.

Service	Responsibilities
User Service	User profile management, authentication & authorization (AuthN/AuthZ), address book.
Product Catalog Service	Product data, category management, search.
Order Service	Order creation, order lifecycle management, order history.
Inventory Service	Stock management, and reserving/releasing inventory in sync with order state.
Payment Service	Payment orchestration across Stripe and PayPal, and refund processing.
Shipping Service	Carrier integrations with FedEx and UPS, and shipment tracking.
Notification Service	Email, SMS, and push notifications, driven by templates and platform events.

Because each service owns a specific business capability, teams can deploy, scale, and evolve them independently — for example, the Inventory Service can scale up during a flash sale without needing to scale the Notification Service.

6. Event Streaming & Message Broker

Rather than having microservices call each other directly, the platform uses asynchronous, event-driven communication. This decouples services so that a slowdown or failure in one does not cascade into others.

- **Azure Event Hubs** – ingests high-volume event streams (e.g., clickstream, telemetry) for downstream processing and analytics.
- **Apache Kafka / Azure Service Bus** – acts as the central message broker between microservices, carrying business events such as:
 - Order Created
 - Payment Succeeded
 - Inventory Reserved
 - Order Shipped

Each microservice publishes events when its own state changes and subscribes to the events it cares about from other services. For example, the Order Service publishes “Order Created,” which the Inventory Service consumes to reserve

stock, and the Payment Service consumes to begin payment orchestration — all without those services calling one another synchronously.

7. Data Layer

The platform deliberately uses different storage technologies for different data shapes and access patterns (polyglot persistence), rather than forcing everything into one database:

Store	Technology	Used For
Operational Databases	Azure SQL Database	Orders, users, and payments; chosen for ACID transactions and strong consistency.
Product Catalog DB	Azure Cosmos DB	Products and categories; globally distributed for high availability and low-latency reads worldwide.
Cache	Azure Cache for Redis	Session data, cart contents, and other frequently accessed data, to reduce database load and latency.
Blob Storage	Azure Blob Storage	Product images, invoices, and documents.
Data Lake	Azure Data Lake	Analytics data, logs, and raw event data for downstream processing.

The message broker sits directly above the data layer in the architecture, reflecting that many of these stores are updated in response to events rather than through direct, synchronous writes from every calling service.

8. External Integrations

The platform integrates with several categories of third-party services, all accessed from the microservices layer:

- **Payment Gateways** – Stripe and PayPal handle payment capture, authorization, and settlement.
- **Shipping Carriers** – FedEx and UPS provide rate calculation, label generation, and delivery tracking.
- **Communication** – Email, SMS/push, and WhatsApp channels deliver order and account notifications.
- **Other Services** – a Tax Service calculates jurisdiction-specific tax, and a Fraud Detection service screens transactions for risk.

9. Analytics & Observability

Operational and business data flows into a dedicated analytics and observability layer, so both engineering and business teams can understand system health and customer behavior:

- **Azure Synapse Analytics** – powers the analytics dashboard used by administrators (the final step of the user flow).
- **Azure Monitor & Application Insights** – provides monitoring and alerting across the microservices and infrastructure.
- **Azure Log Analytics** – centralizes logs and diagnostics for troubleshooting and auditing.

10. Platform Services – Security, DevOps & Governance

A foundational layer of platform services underpins every other layer in the architecture:

- **Azure Active Directory** – manages identity and access across the platform.
- **Azure Key Vault** – centralizes secrets management (API keys, connection strings, certificates).
- **Azure DevOps** – runs CI/CD pipelines for building, testing, and deploying services.
- **Azure Container Registry** – stores and manages container images used by AKS.
- **Azure Backup** – provides backup and disaster recovery.
- **Azure Policy & Governance** – enforces compliance and organizational standards across resources.

11. Key Architectural Attributes

- **Highly Scalable** – microservices run on AKS with auto-scaling, so capacity grows and shrinks with demand.
- **Fault Tolerant** – multi-AZ deployment, retries, and circuit breakers prevent single points of failure from taking down the platform.
- **Real-time Updates** – the event-driven architecture propagates state changes (like order status) with minimal delay.
- **Cost Efficient** – right-sized services, caching, and CDN usage reduce unnecessary compute and bandwidth costs.
- **Secure** – end-to-end security is enforced via WAF, Key Vault, and identity controls at every layer.

12. Putting It All Together – A Single Order's Journey

To see how the layers connect in practice, consider a single customer order:

- **1. Entry** – the customer's request from the mobile or web app passes through Azure Front Door, CDN, WAF, and API Management.
- **2. Order creation** – API Management routes the request to the Order Service, which creates the order and publishes an “Order Created” event to Kafka/Service Bus.
- **3. Downstream reactions** – the Inventory Service reserves stock and the Payment Service orchestrates payment through Stripe or PayPal, each reacting to events rather than being called directly.
- **4. Persistence** – order and payment data lands in Azure SQL, catalog reads come from Cosmos DB, and hot data is served from Redis.
- **5. Fulfillment** – once payment succeeds, the Shipping Service engages FedEx or UPS, publishing an “Order Shipped” event that the Notification Service turns into an email, SMS, or push update to the customer.
- **6. Visibility** – throughout the process, logs and metrics flow into Azure Monitor, Log Analytics, and Synapse, giving administrators real-time analytics and giving engineers the observability needed to keep the system fault-tolerant.

This combination of edge protection, decoupled microservices, event-driven communication, polyglot storage, and strong platform-level governance is what allows the system to reliably support millions of orders per day while remaining scalable, fault-tolerant, and cost-efficient.